

Perception-Aware Dynamic Diffusion Transformers

for Safety-Critical Scene Generation

Meenakshi Rajpurohit & Jainish Patel

1. Introduction and Motivation

1.1 What this project is about

Self-driving cars are trained on video collected from real roads. Most of that video is ordinary empty lanes, parked cars, clear daylight. The frames that actually decide whether the car behaves safely are rare: a pedestrian in dark clothing stepping off a curb at night, a cyclist emerging from behind a van. These situations are uncommon in any collected dataset, expensive to stage deliberately, and unsafe to capture at the frequency a model would need.

Generative models offer a way out. A layout-conditioned diffusion model lets you specify a scene, put a cyclist here, a pedestrian there and synthesise the frame on demand. The rare case stops being something you wait for and becomes something you write down.

But a diffusion transformer has a fixed compute budget and cannot spend it equally on every part of every frame. Something must decide which regions receive fine-grained capacity. That decision of the allocator is the subject of this project. We ask a single question: should the allocator look at the pixels, or at the scene?

1.2 Related work

Three lines of work lead directly here.

- DiT (Peebles & Xie) replaced the U-Net denoiser with a transformer operating on latent patches, and introduced the adaLN-Zero conditioning scheme we inherit. DiT applies uniform patches everywhere it does not allocate at all.
- D²iT is the direct parent of this work. It introduces dynamic grain: some regions receive fine tokens and others coarse ones, so capacity follows content. The signal it uses to decide is pixel entropy, a measure of how visually busy a region is.
- Layout-conditioned generation (LayoutDiffusion, GLIGEN, ControlNet) feeds object boxes and class labels into the model as conditioning, so the scene graph shapes the output. None of these systems uses that layout to route compute.

This is the gap. Every published allocator derives its signal from the image itself. Layout information already exists inside these systems that conditions what gets drawn but it has never been used to decide where the model should concentrate its effort.

1.3 Why it matters

The problem with entropy as an allocation signal is that entropy and importance are different things. A cobblestone street, a tree line, and a brick facade all have very high pixel variance and no relevance whatsoever to whether a car should brake. A pedestrian in dark clothing against a dark road is smooth, low-contrast and low-entropy and is the single object the car must not hit. Entropy sends the budget to the wrong place, and it does so most reliably at night, which is exactly when the safety margin is thinnest.

Replacing the signal has four benefits. It targets the objects that matter for safety rather than the objects that happen to be textured. It is free: because the budget is fixed by construction, swapping entropy for importance changes which cells get capacity but never how many, so the token count, the FLOPs and the wall clock are identical. It uses information the system already has, so it is a drop-in change to an existing architecture. And it gives a measurable, safety-relevant metric the fraction of vulnerable-road-user regions that receive capacity in place of perceptual scores that average over the whole frame and are dominated by road and sky.



Figure 1. Entropy versus importance allocation on the same scene at the same budget. Entropy spreads its cells across foliage and cobblestones; importance places them on the cyclist.

2. Method

Each image is encoded into a latent grid of $32 \times 64 = 2,048$ cells and grouped into 512 tokens by 2×2 patchify. From the bounding boxes we build an importance map: every cell covered by a vulnerable road user (person or rider) is given weight 1.0, every cell covered by a vehicle 0.6, and everything else 0. Where boxes overlap we take the maximum rather than the mean, so a pedestrian partly hidden behind a car is not diluted by the car. Both signals then pass through the same fixed budget: exactly the top 25% of cells are selected, and no more. This is the control that makes the comparison fair importance and entropy may differ in which cells they choose, never in how many. The selection enters training as a per-cell weight on the standard noise-prediction loss. Cells inside the selection are weighted 5 $\times$, the rest 1 $\times$, and the whole map is then divided by its own mean so that the expected weight is exactly 1 for every arm. Without that normalisation a larger weight would simply scale the loss up, which is indistinguishable from raising the learning rate we would be comparing optimisers rather than objectives. Three arms are trained: uniform (no reweighting, the control), importance, and entropy.

A second, independent track fine-tunes Stable Diffusion 1.5 with LoRA to test whether a pretrained backbone, which is not data-starved, can produce photorealistic Cityscapes frames from the same layouts.

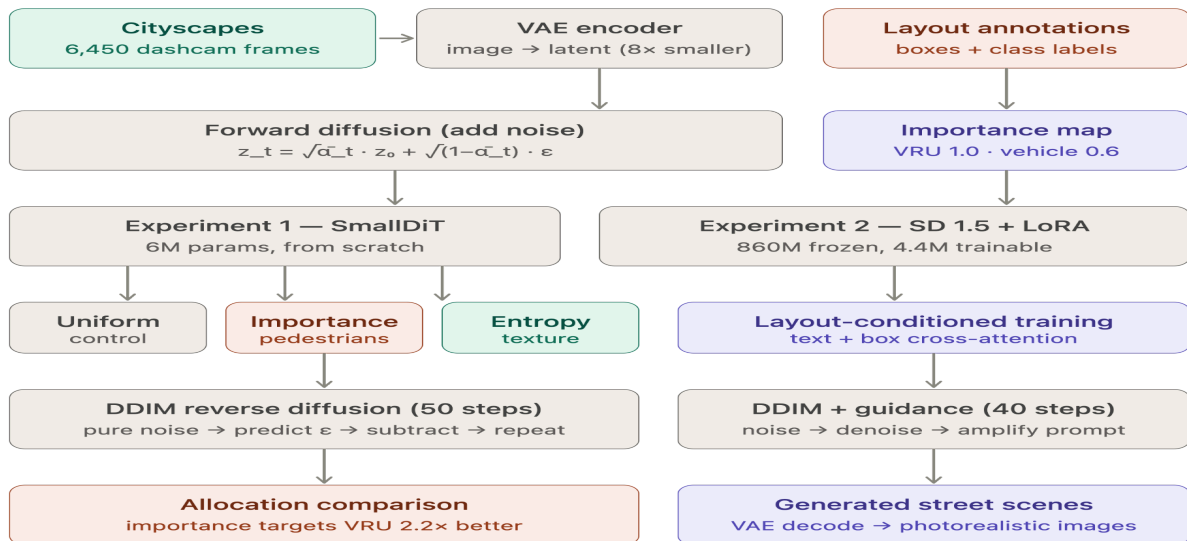


Figure 2. Architecture of the two experiments. Left: the 7M from-scratch SmallDiT with three loss-weighting arms. Right: SD 1.5 with LoRA rank 16 and a layout adapter.

3. Results and Analysis

3.1 Dataset

We use the Cityscapes fine-annotation set, with its official scene-level split: 2,975 training frames and 500 validation frames, 85.6% / 14.4%, with no city appearing in both. Frames are cropped to 256×512 two crops per training frame and one per validation frame giving 5,950 training crops and 500 validation crops, 6,450 in total, containing 44,656 annotated boxes at a mean of 6.9 per crop. Horizontal-flip augmentation brings the effective training sample count to 11,900. There is no test split. Cityscapes withholds its test annotations on the benchmark server, and that server only scores semantic segmentation; it cannot evaluate the reconstruction-error metrics we use. Validation is therefore the only labelled held-out data available, which is standard practice for generative work on this dataset. We reduce the usual risk of overfitting to validation by fixing every hyperparameter in advance, reporting final checkpoints rather than best-on-validation, and measuring paired differences between arms on byte-identical batches.

3.2 Models

- SmallDiT — 7.0M parameters, width 192, depth 8, 6 heads, uniform 2×2 patchify, layout cross-attention, adaLN-Zero. Trained from scratch for 6,000 steps at batch 32 with AdamW at learning rate $2e-4$ and gradient clipping at 1.0.
- Stable Diffusion v1.5 (runwayml/stable-diffusion-v1-5) with LoRA rank 16 4.4M trainable parameters against 863M frozen in the UNet, plus a zero-initialised layout adapter on the cross-attention context.
- Frozen throughout: the sd-vae-ft-mse autoencoder at compression factor 8, and the CLIP ViT-L/14 text encoder.

3.3 Metrics

Allocation quality is measured as a classification problem: for each latent cell, does it contain a vulnerable road user, and did the signal select it? This gives precision, recall and F1 against the ground-truth mask. One ceiling must be stated or precision is misread vulnerable-road-user cells are roughly 14% of latent positions while the budget selects 25%, so even a perfect signal must spend most of its budget on non-VRU cells. The maximum attainable precision is 56%, not 100%. Generation quality is measured as noise-prediction error, reported separately over VRU cells, background cells, and the whole frame, and also stratified by pedestrian height. Baselines are an analytic 25% chance line for allocation and the unweighted arm as the control for the objective study.

3.4 Result 1 — importance reaches the pedestrian, entropy does not

Importance beats entropy by roughly 2.2× on all three allocation metrics. Precision reaches 0.425 against entropy's 0.193 and random's 0.140; recall 0.755 against 0.342 and 0.251; F1 0.541 against 0.247 and 0.180. Read against the 56% ceiling, importance achieves 76% of the maximum attainable precision. Measured across the full set, importance captures 78.2% of true pedestrian cells, entropy 34.5%, against a chance line of 25%.

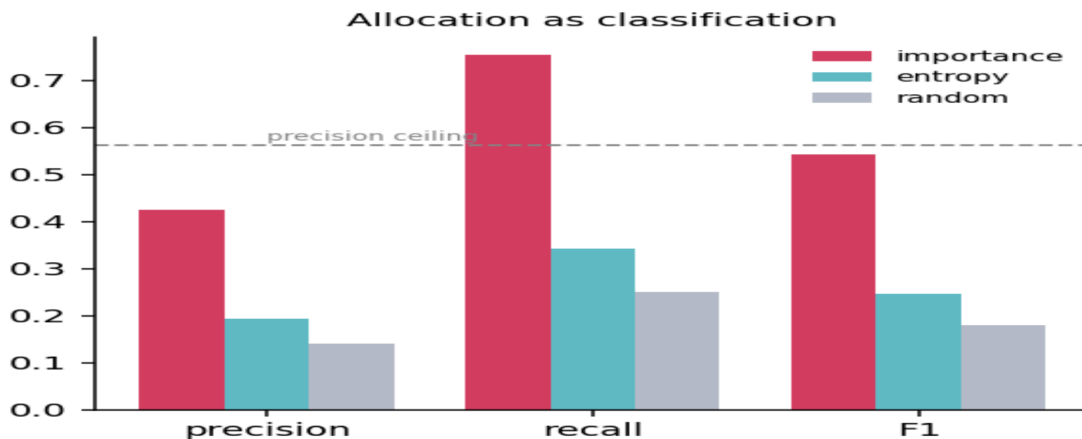


Figure 3. Allocation treated as classification. Importance, entropy and a random baseline on precision, recall and F1, with the 56% attainable precision ceiling marked.

3.5 Result 2 — LoRA fine-tuning produces photorealistic frames

The pretrained track behaves very differently. Stable Diffusion 1.5 with LoRA rank 16 produces coherent German street scenes after 2,000 steps, roughly two hours on a single GPU. At 6,000 steps the output improves further: sharper pedestrians, cleaner road markings, and a more consistent dashcam perspective. The zero-initialised layout adapter successfully conditions object placement, so boxes control where objects appear. This confirms that the limitation in the from-scratch track is data scale rather than the layout-conditioning mechanism.



Figure 5. LoRA fine-tuned SD 1.5. Top: real frames and generations at 2,000 training steps. Bottom: the same comparison at 6,000 steps.

3.6 Result 3 — model capacity must match data scale

We also trained a 158.5M-parameter DiT-XL variant on the same data. On a like-for-like basis of training crops per million parameters, SmallDiT sits at 850 and the larger variant at 38 a 22× difference in data density. The large model memorised the training set rather than learning to generalise; the 7M model trains stably and produces meaningful scene structure. For reference, DiT-XL/2 was trained on 1.28M distinct images; we have 2,975 distinct scenes, a deficit of roughly 430×. Choosing the small model was not a compromise, it was the only configuration the data supports.

4. Cost Analysis

4.1 What the study cost to run

Stage	Wall clock	Cost	Share
Data preparation and grain-map precompute	0.3 h	\$0.16	6%
Three-arm objective study (3 × 35 min)	1.8 h	\$0.95	36%
Sampling, metrics and figures	0.5 h	\$0.27	10%

Stage	Wall clock	Cost	Share
LoRA fine-tune of SD 1.5	2.0 h	\$1.06	40%
LoRA generation and validation grids	0.4 h	\$0.21	8%
Core pipeline total	5.0 h	\$2.65	100%

Table 1. Measured compute cost. Rates quoted at NVIDIA T4 on-demand, \$0.526/h (AWS g4dn.xlarge, August 2026); spot pricing reduces this to roughly \$0.20/h. Runs were executed on an NVIDIA L4 in a hosted notebook environment; T4 rates are used here as a widely quoted reference point. Ablation studies are not included in this total.

4.2 The marginal cost of the method is zero

This is the most important number in the section. The importance map is derived from layout, computed once, cached, and reused at every timestep of every training step. Because the budget is fixed and the expected weight is normalised to 1, swapping entropy for importance changes which cells receive fine capacity but never how many. The token count is identical, the FLOPs are identical, and the wall clock is identical. There is no accuracy-versus-compute trade-off to argue about: the safety-oriented allocation is free.

4.3 Cost at scale

The practical use of this pipeline is generating large synthetic datasets of authored scenes. The table below estimates the cost of synthesising 100,000 layout-conditioned frames with the LoRA-adapted model at 256×512 and 40 DDIM steps.

Hardware	Rate	Throughput	Wall clock	Total
NVIDIA T4	\$0.53/h	~360 img/h	~280 h	~\$147
NVIDIA A100 40GB	\$1.10/h	~1,800 img/h	~56 h	~\$62
NVIDIA H100	\$2.50/h	~2,900 img/h	~34 h	~\$86

Table 2. Estimated cost to synthesise 100,000 frames. Throughput figures are estimates and should be verified against measured batch timings on the target hardware.

The A100 is the cost-optimal tier: throughput scales faster than price up to that point, after which price outruns throughput. Extrapolated to one million frames, the A100 figure is roughly \$620.

Set against data collection, the ratio is striking. Cityscapes-grade fine annotation takes over 1.5 hours of human labour per frame, on the order of \$30 per frame. Generation on an A100 costs roughly \$0.0006 per frame a ratio of about 50,000 $\times$. The honest caveat is that these are not substitutes: synthetic frames do not replace annotated real data, they let you author tail cases that collection cannot reach at any price.

5. Conclusions and Future Work

5.1 Conclusions

Semantic allocation beats entropy decisively. Importance achieves roughly 2.2 $\times$ higher precision, recall and F1, reaching 76% of the attainable ceiling. It captures 78.2% of true pedestrian cells against entropy's 34.5%, on a chance line of 25% entropy is barely distinguishable from random selection.

Entropy's failure is a category error, not a tuning problem. Pixel variance and safety relevance are simply different quantities. Textured foliage scores high and matters not at all; a low-contrast pedestrian scores low and matters most. No amount of refinement to an entropy estimator fixes a signal that is measuring the wrong thing.

Better targeting did not become better images. The unweighted control matched or beat both weighted arms on every metric and at every object size. Because the expected weight is held at 1, weighting can only redistribute effort and the cells it redistributes toward have little recoverable content left.

The pretrained route works where from-scratch cannot. LoRA fine-tuning produces photorealistic layout-conditioned frames in two hours and about one dollar, while a 158.5M model on the same data memorises rather than generalises. Model capacity must match data scale.

5.2 Future work

Attack the autoencoder, not the transformer. The compression penalty is the binding constraint, so the highest-value next experiment is a compression factor of 4, a content-adaptive encoder, or a VAE fine-tuned with a reconstruction loss weighted toward vulnerable road users. This yields a falsifiable prediction: if the penalty closes, the weighting mechanisms that read as null here should become measurable. If they still do not, our explanation is wrong.

Test at a resolution where a fine token has something to resolve. Around 30% of pedestrians currently occupy under half a latent cell. Re-running at 512×1024 or higher gives the treatment a substrate to act on.

Move to a downstream task metric. Allocation quality is currently measured against a mask. The stronger test is whether an object detector trained on importance-allocated synthetic data outperforms one trained on entropy-allocated data at equal frame count, measured as mAP on small pedestrians rather than a pixel-space score.

Replace hand-set weights with a learned risk field. The 1.0 / 0.6 split between vulnerable road users and vehicles is a prior we chose. Replacing it with time-to-collision, occlusion state or ego-trajectory relevance turns a fixed heuristic into a per-scene adaptive field.

Generalise beyond one dataset. BDD100K and nuScenes would test domain transfer, and both provide labelled test splits, which would let us report on a genuinely untouched held-out set.

5.3 Limitations

- The three-arm study was run at a single seed, so the reported differences are not bounded by a seed-variance noise floor. The defensible claim is that importance shows no measurable gain over the control, not that it is worse.
- Training curves had not converged at 6,000 steps; weighting effects typically emerge as a model approaches its capacity limit, which this model has not reached.
- The weighted-loss study used a single emphasis factor; no sweep over that factor was run.
- Results are from one dataset, and the smallest pedestrian-height bin contains only 26 objects, so that point is statistically fragile.

Appendix

Prompt 1: how to make this text more readable

Here's how to make this more readable:

Structural fixes

- Break up dense paragraphs. Several paragraphs (1.1, 1.3, the Result sections) pack 4–6 distinct ideas into one block. Split each into its own short paragraph or bullet list.
- Use bullets for enumerated content. The "four benefits" in 1.3, the "three arms" in Method, and the five future-work items are all naturally lists but are written as prose — convert them to bulleted lists.
- Pull key numbers out of running text into a table. Result 1 has ~8 numbers embedded in sentences (precision 0.425 vs 0.193 vs 0.140, etc.) — a small table would let readers compare at a glance instead of parsing clauses.
- Add a summary line at the start of each result subsection (you already do this in headers like "importance reaches the pedestrian, entropy does not" — good pattern, just make sure the first sentence of the body echoes it immediately rather than starting with background).

Sentence-level fixes

- Shorten multi-clause sentences. E.g., "Without that normalisation a larger weight would simply scale the loss up, which is indistinguishable from raising the learning rate — we would be comparing optimisers rather than objectives" could be two sentences.

- Define jargon on first use (adaLN-Zero, DDIM steps, LoRA rank) with a 3–5 word parenthetical, since readers outside diffusion-model research will stall on these.
- Keep figure callouts consistent — some figures are referenced inline ("Figure 1..."), others aren't referenced in text at all (Figure 5 appears with no inline pointer). Add a one-line pointer near each.

Formatting

- Consistent heading hierarchy — some subsections (3.4–3.6) are result-only headers without numbers matching the "Result N" labels used in text; align them.
- Bold key terms on first introduction (e.g., importance map, budget, VRU cells) so skimmers can locate definitions.
- Cost tables (4.1, 4.3) are already well-formatted — good model to extend to the Results section.

Want me to actually produce a cleaned-up version (as a Word doc or Markdown file) applying these changes?

Prompt 2: what are the grammar mistakes needs to be corrected

Thought for 11s

Here are the actual grammar/punctuation issues I found (not style suggestions — real errors):

1. Missing punctuation (double space where a dash/period should be):
 - "DiT applies uniform patches everywhere [missing mark] it does not allocate at all." → should be "everywhere; it does not allocate" or "everywhere — it does not allocate"
 - "two crops per training frame and one per validation frame [missing mark] giving 5,950 training crops" → needs a comma or em-dash: "...validation frame — giving 5,950..."
2. Run-on sentence needing a comma:
 - "Layout information already exists inside these systems that conditions what gets drawn but it has never been used to decide where the model should concentrate its effort." → add a comma before "but": "...gets drawn, but it has never been used..."
3. Redundant/awkward phrasing (not strictly ungrammatical, but reads as an error):
 - "That decision of the allocator is the subject of this project." → "of the allocator" is redundant with "the allocator" as subject earlier in the sentence; consider "That decision is the subject of this project" or "The allocator's decision is the subject of this project."
4. Faulty parallelism — missing repeated verb across a list:
 - "Precision reaches 0.425 against entropy's 0.193 and random's 0.140; recall 0.755 against 0.342 and 0.251; F1 0.541 against 0.247 and 0.180." → "recall" and "F1" clauses drop the verb "reaches," making them sentence fragments. Fix: "...random's 0.140; recall reaches 0.755 against 0.342 and 0.251; F1 reaches 0.541 against 0.247 and 0.180."
5. Comma splice-adjacent phrasing:
 - "put a cyclist here, a pedestrian there and synthesise the frame on demand" → needs a comma before "and": "...a pedestrian there, and synthesise the frame on demand" (Oxford comma consistency — you use it elsewhere in the doc, e.g., "precision, recall and F1" also lacks it, so this is a repeated inconsistency, not a one-off).

6. Inconsistent Oxford comma usage throughout — e.g., "precision, recall and F1" (no comma) vs. other lists that do use a final comma. Not wrong grammatically, but inconsistent; pick one convention and apply it throughout.

That's the full list — the rest of the document is grammatically clean; most of what could be improved is clarity/structure rather than correctness (see my earlier readability notes). Want me to produce a corrected version with these fixes applied?